IoT-Based Home Automation System

Title:IoT-Based Home Automation System
Submitted To: ISTE Project Competition Committee
Institution:CSMSS Chh. Shahu College of Engineering, Chh.Sambhaji Nagar
Department :Electronics & Communication (ACT)
Submitted by : Kishor Vitthal Kakde

1. Introduction

Home automation using IoT (Internet of Things) enhances the quality of life by enabling smart control of electrical appliances via smartphones and voice assistants. This project leverages affordable microcontrollers, wireless communication, and IoT cloud platforms to create an intelligent and scalable home automation system.

2. Objectives

* Automate control of home appliances (lights, fans, etc.) using IoT.
* Enable remote monitoring and operation through a mobile app.
* Utilize sensors for environment-responsive automation.
* Improve energy efficiency and user convenience.

3. System Architecture

The system is built around a **Raspberry Pi** as the central controller, with connected relays to switch appliances. Devices communicate via **Wi-Fi.** A mobile application or web interface communicates with the Raspberry Pi through the cloud.

Block Diagram:

- Raspberry Pi (Controller)
- Relay Module
- Sensors (e.g., PIR, Temperature, Humidity)
- Wi-Fi Router
- Mobile App(Firebase database)

4. Components Used

- Raspberry Pi 3B– Central control unit.
- Relay Module – Controls AC appliances.
- DHT11 Sensor – Measures temperature and humidity.
- PIR Sensor– Detects human motion.
- Smartphone– Interface via Firebase database or custom app.
- Wi-Fi Router – Enables cloud access.

5. Working & Implementation

- The Raspberry Pi hosts a Python program that receives input from the mobile app and sensors.
- When a user activates a switch from the app, the Raspberry Pi triggers the corresponding relay.
- Sensor inputs like motion or temperature can automate responses (e.g., turning on fans when it gets hot).
- Data can also be logged and monitored via the cloud.

6. Advantages

- Remote access and control.
- Increased home safety through automation.
- Customizable rules and scheduling.
- Energy-efficient operation.

7. Limitations

- Dependent on stable internet connectivity.
- Initial setup cost of components.
- Security risks if not properly configured.

8. Applications

- Smart lighting and climate control.
- Elderly and differently-abled assistance.
- Energy usage analytics.
- Security systems (motion detection, door sensors).

9. Future Scope

- Integration with AI for behavior learning.
- Voice assistant support (e.g., Alexa, Google Assistant).
- Expandable to offices and industrial automation.
- Solar-powered and offline backup capability.

10. Conclusion

This project demonstrates a cost-effective and efficient way to bring smart automation into homes. By using open-source platforms and IoT hardware, it creates a foundation for more intelligent and connected living spaces.

References:

1. Raspberry Pi Documentation - https://www.raspberrypi.org
2. Blynk IoT Platform - https://blynk.io
3. DHT11 Sensor Data Sheet
4. Python GPIO Libraries
5. IEEE Journals on IoT and Home Automation

IOT- Based Device Control Using Raspberry Pi and Firebase (Home Automation)

Name: [Kishor Vitthal Kakde]
Roll Number: [AC2109]
Department: [ELECTRONICS AND COMMUNICATION(ACT)]
College: [CSMSS,CSCOE CHH. SAMBHAJI NAGAR]

Introduction :-
This project demonstrates how to control an electrical device (such as an LED or a relay) connected to a Raspberry Pi using Firebase as a cloud-based interface. It uses Python programming, GPIO for hardware control, and Firebase Realtime Database to enable remote control of devices via the internet.

Objective :-
To implement a system where a device can be turned ON or OFF remotely using data stored in Firebase, making it suitable for IoT-based automation systems.

Explanation of the Code:

Explanation of the Code

```
from firebase import firebase
import RPi.GPIO as GPIO
from time import sleep

GPIO.setmode(GPIO.BOARD)
GPIO.setup(40, GPIO.OUT)

firebase =
firebase.FirebaseApplication('https://iotproject-f5403.firebaseio.com/',
None)

while True:
    sleep(1)
    result = firebase.get('Status', None)
    if result == 1:
        GPIO.output(40, 1)
        print("ON")
    if result == 0:
        GPIO.output(40, 0)
        print("OFF")
```

Code Breakdown:-
firebase.FirebaseApplication(...): Connects to the Firebase Realtime Database.
GPIO.setmode(GPIO.BOARD): Uses physical pin numbering.
GPIO.setup(40, GPIO.OUT): Sets pin 40 as output.
Inside a loop, it reads the value of Status from Firebase every second.
If Status is 1, it sets pin 40 HIGH (ON); if 0, LOW (OFF).

Working of the System:-
The user changes the 'Status' value in the Firebase database (via mobile or web).
Raspberry Pi checks this value every second.
GPIO pin is turned ON or OFF based on the value.

Applications:-
- Home Automation (Fan/Light control)
- Smart Agriculture (Pump control)
- Industrial IoT
- Remote energy control systems

Advantages:-
- Real-time control from anywhere
- Easy to modify and scale
- Uses free cloud service (Firebase)

Conclusion:-
This project successfully demonstrates how Raspberry Pi can act as an IoT device by remotely controlling hardware using cloud services. The integration of Firebase allows real-time and reliable operation over the internet.

Future Scope:-
- Add multiple device controls
- Real-time status update and feedback
- Integrate voice assistants (Google Assistant, Alexa)
- Add sensors for automation (temperature, motion, etc.)

Raspberry Pi 3 Model B:-
The Raspberry Pi 3 Model B is a compact, powerful computer used for various embedded and IoT projects. It features a 64-bit quad-core CPU, onboard Wi-Fi and Bluetooth, GPIO pins, and multiple connectivity ports.
Extra information about Raspberry Pi 3 Model B:-
The Raspberry Pi 3 Model B is a credit card-sized single-board computer ideal for embedded systems and IoT applications. In your IoT-Based Home Automation System, it serves as the central controller, managing communications between the mobile app and connected appliances via GPIO and Wi-Fi.

Key Specifications:-
CPU: 1.2 GHz 64-bit quad-core ARM Cortex-A53
RAM: 1 GB LPDDR2

Wireless Connectivity:-
Wi-Fi: 802.11n (2.4 GHz)
Bluetooth: 4.1 (BLE support included)

Ports:
4 × USB 2.0
HDMI (for display output)
3.5 mm audio jack with composite video
Ethernet port (10/100 Mbps)
CSI (Camera Serial Interface)
DSI (Display Serial Interface)

GPIO Pins:
40-pin header with multiple GPIOs, PWM, I^2C, SPI, and UART

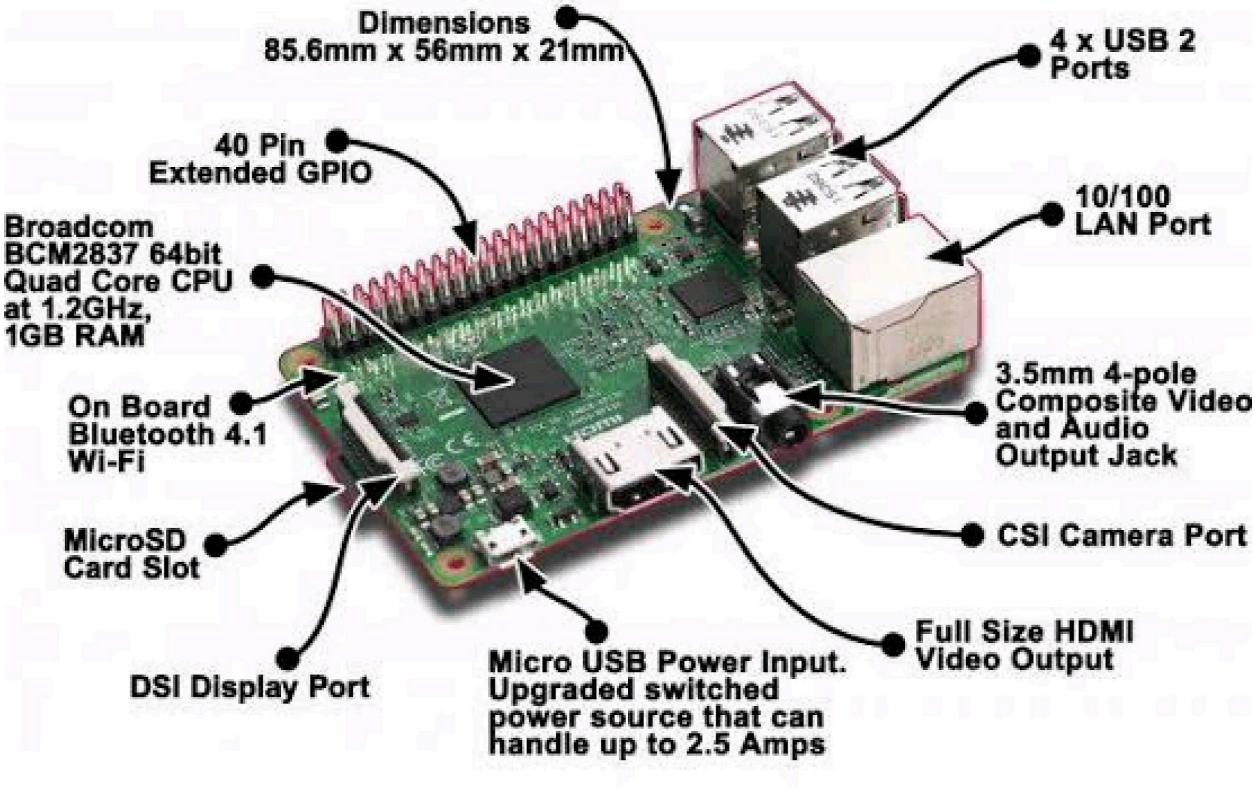
Storage:
microSD card slot (boot and primary storage)

Role in Home Automation:
Controller: Receives commands from a mobile app over Wi-Fi
GPIO Control: Activates relays connected to appliances (e.g., lights, fans) through GPIO pin 14
Internet Integration: Allows remote access and control of devices from anywhere
Expandable: Supports additional modules like sensors, cameras, or voice assistants (e.g., Google Assistant)

Advantages in IoT Projects:
Low power consumption
Built-in wireless connectivity
Affordable and community-supported
Compatible with Python, Node.js, and other programming languages
Easily integrates with home automation platforms like Home Assistant or OpenHAB

Key Components:
- 40-Pin GPIO Header
- USB Ports (x4)
- HDMI Port
- Ethernet Port
- Wi-Fi and Bluetooth
- MicroSD Card Slot

- Micro USB Power Port.
- FIGURE:-



Relay Module

A relay is an electromechanical switch used to control high-voltage devices with a low-voltage signal from the Raspberry Pi.

Relay Pins:

- VCC: Power (5V)
- GND: Ground
- IN: Signal pin
- NO: Normally Open
- NC: Normally Closed
- COM: Common terminal

Working: When Raspberry Pi sends a HIGH signal to IN pin, COM connects to NO, turning ON the device.

End of Report